

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..21

UI COMPONENTS

 Copy as Markdown

Authentication UI components

The authentication module ships with UI components and controllers for building sign-in interfaces. This page covers the all-in-one `SignInWidget`: how to control which providers it shows, how to style its buttons, and how to translate its texts. Each provider also has its own widgets and controllers for building a custom UI, covered on the provider pages.

SignInWidget

The `SignInWidget` is an all-in-one widget that automatically detects available authentication providers and displays the appropriate sign-in options. The [setup page](#) shows the full wiring. In short:

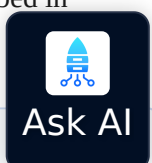
```
SignInWidget(  
  client: client,  
  onAuthenticated: () {  
    // Runs after a successful sign-in. Do not navigate here; listen to  
    // authentication state changes instead.  
  },  
  onError: (error) {  
    // Show the error to the user.  
  },  
)
```

Disabling providers

You can hide specific providers in the app while keeping them available on the server. This is useful when you want to phase out a provider but keep compatibility with older app versions.

The reverse is not possible. A provider only appears when it is configured on the server, as described in [identity providers configuration](#).

```
SignInWidget(  
  client: client,
```



```

disableEmailSignInWidget: false,
disableGoogleSignInWidget: false,
disableAppleSignInWidget: true, // Disable Apple sign-in
onAuthenticated: () {
  // Do something when the user is authenticated.
  //
  // NOTE: You should not navigate to the home screen here, otherwise
  // the user will have to sign in again every time they open the app.
},
)

```

Customizing SignInWidget

The `SignInWidget` builds a default widget for each identity provider it detects. Pass your own widget for a provider to replace the default one:

```

final signInWidget = SignInWidget(
  client: client,
  emailSignInWidget: EmailSignInWidget(
    client: client,
    startScreen: EmailFlowScreen.login,
    // NOTE: When you opt-out of the internal widget, you need to provide your
    // own `onAuthenticated` and `onError` callbacks.
    onAuthenticated: _onAuthenticated,
    onError: _onError,
    // ... custom configuration
  ),
  googleSignInWidget: GoogleSignInWidget(
    client: client,
    scopes: const [
      ...GoogleAuthController.defaultScopes,
      'https://www.googleapis.com/auth/youtube',
    ],
    onAuthenticated: _onAuthenticated,
    onError: _onError,
    // ... custom configuration
  ),
  onAuthenticated: _onAuthenticated,
  onError: _onError,
);

void _onAuthenticated() {
  // Handle successful authentication
}

void _onError(Object error) {
  // Handle errors
}

```

Styling the buttons

Inside `SignInWidget`, every provider button shares one neutral, theme-aware appearance. To change it, pass a `SignInButtonStyle` as `buttonStyle`:

```
SignInWidget(
  client: client,
  buttonStyle: const SignInButtonStyle(
    shape: SignInButtonShape.rounded,
    text: SignInButtonTextVariant.signInWith,
  ),
  onAuthenticated: _onAuthenticated,
  onError: _onError,
)
```

Fields set on `buttonStyle` apply to the provider buttons, and they also override the same-named arguments on a custom provider widget you pass to `SignInWidget`. Fields left unset fall through to the widget's own arguments. Brand style presets, such as `GoogleButtonStyle.filledBlack`, only apply when a provider widget is used on its own, outside `SignInWidget`.

For all options of each provider widget, see the provider's customizations page, which also covers building a custom UI with the provider's controller. For example, see [the Google provider](#) or [the email provider](#).

Localization

All text shown by the authentication widgets is in English by default. To replace it, wrap `SignInWidget`, or any individual provider widget, in a `SignInLocalizationProvider` and pass the text objects for the parts you want to translate:

- `BasicSignInTexts` for the shared messages and dividers.
- `EmailSignInTexts` for the email flow: sign-in, registration, account verification, password reset, and the terms and privacy labels.
- `PasswordRequirementTexts` for the labels of the built-in password requirement widgets.
- One text object per provider for the sign-in buttons: `AppleSignInTexts`, `GoogleSignInTexts`, `GitHubSignInTexts`, `MicrosoftSignInTexts`, `FacebookSignInTexts`, and `AnonymousSignInTexts`.

NOTE

The Firebase provider has no texts because it only supplies a controller rather than a finished UI.

The example below shows how to pass every text object explicitly:

```
const basicTexts = BasicSignInTexts(
  noAuthenticationProvidersConfigured:
    'No authentication providers configured',
  orContinueWith: 'or continue with',
);
```

```

const emailTexts = EmailSignInTexts(
  title: 'Sign In with email',
  forgotPassword: 'Forgot password?',
  signIn: 'Sign in',
  dontHaveAnAccount: "Don't have an account?",
  signUp: 'Sign up',
  signUpTitle: 'Sign Up with email',
  continueAction: 'Continue',
  alreadyHaveAnAccount: 'Already have an account?',
  verifyAccountTitle: 'Verify account',
  verifyResetCodeTitle: 'Verify reset code',
  verificationMessage:
    'A verification email has been sent. Please check your email and '
    'enter the verification code below.',
  verify: 'Verify',
  setAccountPasswordTitle: 'Set account password',
  passwordLabel: 'Password',
  backToSignUp: 'Back to sign up',
  setNewPasswordTitle: 'Set new password',
  newPasswordLabel: 'New Password',
  resetPasswordTitle: 'Reset password',
  resetPasswordDescription: 'Enter the email address to reset password.',
  requestPasswordReset: 'Request password reset',
  resetPassword: 'Reset password',
  backToSignIn: 'Back to sign in',
  emailLabel: 'Email',
  termsIntro: 'I have read and accept the ',
  termsAndConditions: 'Terms and Conditions',
  andText: ' and ',
  privacyPolicy: 'Privacy Policy',
);

const passwordTexts = PasswordRequirementTexts(
  minLengthTemplate: 'At least {length} characters',
  maxLengthTemplate: 'At most {length} characters',
  containsLowercase: 'Contains at least one lowercase letter',
  containsUppercase: 'Contains at least one uppercase letter',
  containsNumber: 'Contains at least one number',
  containsSpecialCharacter: 'Contains at least one special character',
);

SignInLocalizationProvider(
  basic: basicTexts,
  email: emailTexts,
  passwordRequirementTexts: passwordTexts,
  apple: const AppleSignInTexts(signInButton: 'Sign in with Apple'),
  google: const GoogleSignInTexts(signInButton: 'Sign in with Google'),
  github: const GitHubSignInTexts(signInButton: 'Sign in with GitHub'),
  microsoft: const MicrosoftSignInTexts(signInButton: 'Sign in with Microsoft'),
  facebook: const FacebookSignInTexts(signInButton: 'Sign in with Facebook'),
  anonymous: const AnonymousSignInTexts(signInButton: 'Sign in as guest'),
  child: SignInWidget(
    client: client,
    onAuthenticated: _onAuthenticated,
    onError: _onError,
  ),
);

```

The `{length}` placeholder in the two length templates is replaced with the configured limit, so keep it in the translated string.

Translating part of the text

Any text object you leave out keeps its built-in English text, so you only need to pass the ones you are translating. To change a few values inside an object, start from its `defaults` constant:

```
SignInLocalizationProvider(  
  email: EmailSignInTexts.defaults.copyWith(  
    title: 'Sign in with email',  
    signIn: 'Log in',  
  ),  
  child: signInWidget,  
);
```

Switching locale

The `SignInLocalizationProvider` widget hands the text objects to the widgets below it. It does not read Flutter's current `Locale` or pick a translation for you. In an app that supports several locales, select the text objects with your existing localization setup and rebuild `SignInLocalizationProvider` when the locale changes.

Related

- [Setup](#): wire the sign-in UI into your app.
- [The basics](#): react to authentication state changes.
- [Custom overrides](#): replace the built-in UI and endpoints entirely.